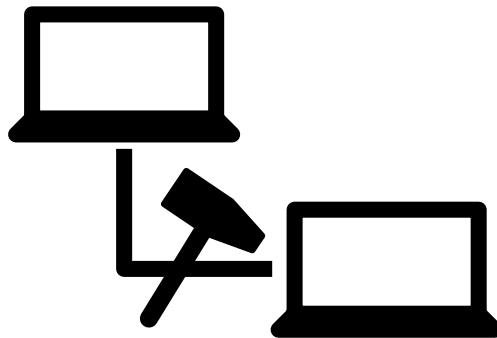


Bachelor Thesis



From Rowhammer to Nethammer Utilizing Intel CAT to Remotely Induce Bit Flips

Lukas Lamster 01404333

Contents

1	Introduction	3
2	Background	5
2.1	DRAM Functionality and Structure	5
2.2	Cache Functionality and Structure	7
2.3	Intel CAT	7
2.4	Rowhammer	8
3	Test Setup	10
4	Reverse-Engineering Properties of DRAM and Memory Controllers	10
4.1	Reverse-Engineering of DRAM Mappings	11
4.2	Reverse-Engineering of Memory Controller Behaviour	12
5	Evaluation of different Rowhammer patterns	14
5.1	Double-sided Hammering	14
5.2	One-location hammering	15
6	Evaluation of Rowhammer with Intel CAT	16
6.1	Double-sided Hammering with CAT	16
6.2	One-location Hammering with CAT	17
7	Nethammer	18
7.1	Initial Considerations	18
7.2	Attack Setup and Evaluation	19
8	Conclusion	20

Abstract

Research showed that accessing DRAM rows with sufficient speed can cause bit flips in adjacent rows. This effect is known as the *Rowhammer* bug. Several previous attacks using this vulnerability relied on flushing the cache of the system under attack with the *clflush* instruction or non-temporal instructions to serve reads directly from DRAM. Various other attacks used eviction sets of different complexity in order to evict aggressor addresses from the cache. In this work, we show that *clflush* and complicated eviction sets are both not necessary in order to mount a Rowhammer attack on certain systems. These results were achieved by attacking systems using Intel cache-allocation-technology (CAT), which is a technology present in Intel server processors. With CAT, the last-level cache can be divided arbitrarily among different classes of service in order to avoid performance issues caused by “noisy neighbors” on cloud computers or servers. Due to the reduction of cache size, it is possible to precisely evict addresses by just one additional memory access, thus, enabling us to serve accesses directly from DRAM. In the course of our research, we greatly improved the performance of a previously published automated reverse-engineering tool [1]. This tool is used to gain knowledge about the mapping functions from physical addresses to memory banks. This knowledge is required to perform memory accesses targeting specific rows in the DRAM. Furthermore, we created a tool with which the row policy of a target system can be reverse-engineered automatically. Combining all previous results, we were able to perform Rowhammer attacks over the network by flooding the target with UDP packets. Parts of this work are included in a paper by Lipp et al. [2].

1 Introduction

In 2014, Kim et al. [3] discovered that reading from DRAM with sufficient speed can alter the content stored in memory cells without writing to them. This effect was dubbed the *Rowhammer bug* and has been studied over the past few years. Based on these findings, Seaborn and Dullien [4] showed that the induced bit flips can be utilized to escape Google Chrome’s NaCl sandbox and also to gain root privileges on a target machine by flipping bits in page-table entries. While these attacks rely on an instruction to flush the target addresses from the cache, to serve read accesses directly from DRAM, Gruss et al. [5] showed that equal results can be achieved with regular memory accesses if they are performed such that the aggressor address is evicted from the cache. Furthermore, they showed that this kind of attack could be performed through JavaScript code, implying that a Rowhammer attack can be mounted over a website. Several countermeasures have been proposed and partially been deployed to mitigate possible Rowhammer attacks [6, 7]. In 2017, Aga et al. [8] showed that certain Rowhammer countermeasures can be overcome if the target system utilizes Intel CAT, a technology that is found in high-end server CPUs. Due to the reduction of the cache size, CAT facilitates Rowhammer attacks which do not need the *clflush* instruction. Still, local code execution is required in these attacks.

In this work, we confirm that CAT enables us to perform Rowhammer attacks without the use of *clflush* by generating a very high cache eviction rate through memory accesses. Furthermore, we show that CAT also allows us to

perform a completely remote Rowhammer attack in which not a single line of attacker code is executed on the target machine. Due to the fact that an explicit cache flush instruction is not required to perform these attacks, restricting the *clflush* instruction is not a sufficient countermeasure to this attack. The remote attack, dubbed *Nethammer*, relies on flooding the target machine with UDP packages. Detecting and rejecting high-bandwidth traffic is not a sufficient countermeasure against this attack either, as the first bit flips can occur after only 350 milliseconds. Thus, a short traffic spike is all that is needed to induce bit flips on the target machine.

To perform a Rowhammer attack using CAT, we had to verify that the given setup is indeed affected by the bug, e.g. by performing a double-sided Rowhammer attack. When using a double-sided hammering pattern, it is crucial to know the mapping from physical addresses to locations on DRAM, as both rows next to the target row must be hammered. Unfortunately, this kind of information is not available; it has to be obtained through reverse-engineering [1]. For this purpose, the DRAMA reverse-engineering tool was modified such that stable measurements were possible on the system under test. With the mapping information, it was possible to perform a double-sided Rowhammer attack on the target system, verifying that the system under test is indeed vulnerable. During testing, we found that the system utilized target-row-refresh (TRR) [9], which is a Rowhammer mitigation technique in which potential target rows are refreshed if neighboring rows are accessed too frequently. We found this countermeasure trivial to circumvent, as bit flips still occur in rows not directly adjacent to the hammered rows. During the process of reverse-engineering the DRAM mapping function, we found that the system under test utilized a so-called *adaptive row policy*, meaning that the memory controller is able to learn certain access patterns, to modify the duration that an accessed row is kept open after the access. As this kind of behavior can affect the chance of a successful Rowhammer attack negatively, we created a tool to reverse-engineer the memory controller behavior in order to determine when the adaptive policy is activated. The gadget was deployed on three different machines such that a comparison between different row policies is possible. Evaluating the results of these measurements, our assumptions about the negative impact of the adaptive policy, especially for single location hammering, were confirmed. We also found that the adaptive policy can be easily avoided by generating enough memory load on the machine.

After validating that regular Rowhammer attacks were working on the system, we moved on to combining the attack with CAT. In this step, all flush instructions were stripped from the code, and the core on which the attacker code was running was limited with CAT. Next, an eviction address was generated for each of the aggressor addresses using the results obtained by Maurice et al. [10]. These eviction addresses are necessary as they are used to flush the aggressor addresses from the cache. With the two aggressor addresses and the two eviction addresses, it is possible to perform a Rowhammer attack without flushing the cache by calling *clflush*. It is also possible to modify this attack such that only one address is hammered. The modified attack is negatively influenced by the adaptive row policy, but generating sufficient memory load solves this problem, yielding results comparable to that of a regular single-sided Rowhammer attack.

With the results gained in these steps, we were able to perform a completely

remote Rowhammer attack via network by flooding the system under attack with UDP packets. Once we enabled CAT on the target system, bit flips were detected while scanning the memory. Increasing the memory load on the target system was found to have a positive influence on the number of detected bit flips.

Outline: The rest of the thesis is structured as follows. Section 2 provides technical background on DRAM, cache, Intel CAT, and Rowhammer. Section 3 describes the laboratory setup which was used in all further tests. In Section 4, we discuss the process of reverse-engineering DRAM and memory controller properties. Section 5 discusses the results of local Rowhammer attacks with *clflush*, including the problems around TRR and how we were able to bypass this mechanism. In Section 6, we discuss the results of flush-free hammering with CAT. Section 7 shows how the Rowhammer attack over the network was implemented. Finally, Section 8 concludes this work.

2 Background

In this Section, we provide background on DRAM functionality and structure, cache functionality and structure, Intel CAT, and Rowhammer.

2.1 DRAM Functionality and Structure

In DRAM, information is stored in so-called *memory cells*. These cells use small capacitors to store electrical charges which are used to represent the state of a bit. The schematic construction of these elements is depicted in Figure 1. As capacitors lose their charge over time, the content of these cells has to be refreshed periodically to remain valid. Multiple of these cells are combined in a so-called *row*. All cells in a row share the same word line. Rows are grouped

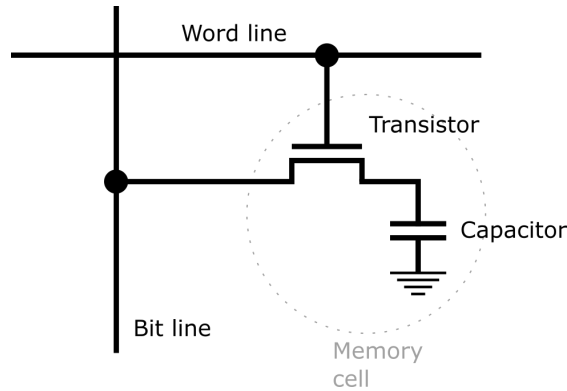


Figure 1: Schematic depiction of a single memory cell

into so-called *banks* in which memory cells at the same row offset in different rows share one bit line. Each bank has a row address decoder which is used to decide which row is accessed (i.e., which word line is activated). If the word line of a row is pulled to a certain voltage level, the row is *activated*, meaning that every memory cell in the row is accessed at the same time. Their content is

made available to the so-called *sense amplifiers*. The purpose of these elements is the amplification of the charge stored in a memory cell. Depending on the stored charge, a logical 0 or 1 is provided to the rest of the circuit. Accessing single bits directly in DRAM is not possible with the structure presented here, as it is only possible to access an entire row. Each memory bank has a so-called row buffer. There are two possible scenarios on memory accesses with regards to the row buffer:

- **Row hit:** If the row buffer holds a row, and the access is to the same row, it is called a *row hit*. In this case, the access is performed using the data stored in the row buffer. Thus, the access latency is small as no data has to be transferred from the target row to the row buffer.
- **Row miss:** If the row buffer holds a row and the access goes to a different row in the same bank, this is called a *row miss* or *row conflict*. Here, the currently open row has to be closed, and the content of the row buffer has to be written back to the cells. Then, the target row has to be opened, and the content of it has to be stored in the row buffer from which it can be transmitted for further use. A row conflict usually takes more than twice as long as a row hit.

On DDR4 modules, the banks described above are grouped into *bank groups* consisting of four banks. These bank groups are then grouped into a *rank* consisting of four bank groups and, thus, of 16 banks overall [11]. On DDR3 modules, there are eight banks per rank and no bank groups [12]. Dual Inline Memory Modules (DIMMs) are the components that are physically plugged into the sockets on the motherboard of the device. A single DIMM can consist of up to four ranks which can be distributed over the two sides of the DIMM. Each DIMM, in turn, is connected to a channel on the motherboard. Several DIMMs can be connected to the same channel. Thus, we get a hierarchical structure with channels at the highest, and rows at the lowest layer.

Row policies: The processor’s memory controller can use three different policies to optimize the latency of memory accesses:

- **Open-row policy:** If an open-row policy is used, memory rows are kept open for a certain amount of time before being closed. The primary motivation behind this policy is the principle of locality. Addresses near the last accessed address are more likely to be accessed in the near future. Thus, keeping the row open will reduce the latency of subsequent memory accesses to the same row.
- **Closed-row policy:** If a closed-row policy is used, memory rows are closed right after the access. This reduces the latency of the next memory access if it goes to a different row in the same bank.
- **Adaptive row policy:** The adaptive row policy tries to combine the advantages of both the closed and the open policy. Here, the row hits and conflicts are counted by the memory controller and subsequently used to dynamically modify the time that accessed rows are kept open. Thus, the number of slow row conflicts caused by keeping a row open too long is minimized. The number of fast row hits, caused by keeping rows open for some time, is increased.

2.2 Cache Functionality and Structure

The cache is a memory residing directly in the CPU. This memory is used to store data which likely is accessed in the near future, with the intention to speed up memory accesses. When a memory access is issued, there are two possibilities regarding the cache:

- **Cache hit:** The accessed data is present in the cache. This scenario is called a *cache hit*. Here, the data is read directly from the cache, no access to main memory is performed. Cache hits are much faster than regular memory accesses going to DRAM. Having a high rate of cache hits is desirable since this yields higher performance.
- **Cache miss:** The data is not present in the cache. This is called a *cache miss*. A cache miss is much slower than a cache hit, as the requested data has to be read from main memory. A high miss rate is not desired as memory operations are slowed down by cache misses.

After reading data from main memory, it is stored in the cache. As the cache has a very limited size, storing data in the cache often means that some other data has to be evicted from the cache in the first place. In state-of-the-art CPUs, we can find 3 to 4 levels of caches. These are named L1, L2, L3 and, if available, L4 cache. The cache level with the highest number is usually denoted LLC, standing for *last level cache*. The L1 cache is the smallest cache and closest to the CPU, hence accesses to this cache are the fastest. Cache capacity increases and access speed decreases with every level. Consequently, the LLC is the cache with the highest storage capacity and also the highest latency. If a cache level contains all data which is also contained in the lower levels, it is called *inclusive*. On most Intel CPUs the L3 is inclusive of all lower cache levels. Thus, it is possible to evict data from lower level cache by evicting it from the LLC.

Caches employ a similar memory layout as DRAM. Data in a cache is stored in so-called *cache ways* which have a capacity of 64 bytes [13]. These ways are grouped into *sets* which consist of n ways, depending on the cache. If data is stored in the cache, the ways of a set are filled with incoming data, the way in which new data will be stored is determined by a cache replacement policy. This policy is usually not publicly known. If every way in a set is occupied and new data should be stored in this set, the CPU's eviction policy selects a cache way from this set whose data is then evicted from the cache, thus providing a place at which the new data can be stored.

Additionally, to sets and ways, the LLC is also divided into so-called *slices*. Each CPU core owns a slice of the LLC, but all slices are connected via a ring buffer, thus making it possible to access all slices from all CPU cores. The location in the cache in which data will be stored depends on the memory address from which the data was loaded. This mapping function is undocumented but can be reverse-engineered, as Maurice et al. [10] have shown.

2.3 Intel CAT

Intel's cache allocation technology (CAT) is a technology which allows restricting access to the LLC for each core separately [14]. CAT can be found in several

Intel server CPUs, starting with the Xeon E5-2600 v3 family which was launched in 2014. The primary motivation behind CAT is the arbitrary distribution of cache space to different CPU cores, such that a process which is over-utilizing the cache will not affect the performance of other processes running on different cores. The number of cache lines available to a specific CPU core can be modified at runtime. This allows a dynamic allocation of cache space which in turn can be used to allocate more resources to high priority applications than to low priority ones. Different configurations of cache distributions can be stored in so-called *classes of service* (CLOS). For each class, a *capacity mask* is used to define the maximum amount of available cache space for this class. The masks of multiple classes of service can overlap, hence applications belonging to two different classes could still share parts of the cache if configured in such a way.

While splitting the cache can eliminate the noise generated by neighbor processes trashing the cache, it also reduces the number of possible cache ways in each cache set, as the number of cache ways is limited per core.

2.4 Rowhammer

Rowhammer is an unwanted effect that occurs when too many accesses to different memory rows in the same DRAM bank are performed in quick succession. As mentioned in Section 2.1, the capacitors used in DRAM lose their charge over time and, thus, have to be refreshed periodically. This leakage of charge can be increased by accessing rows in proximity of a target row (within the same memory bank), which will ultimately result in bit flips in a row if a large amount of memory accesses is performed to nearby rows between two memory refresh cycles. To achieve such a high access rate, an attacker has to make sure that memory accesses are served directly from DRAM and not from the cache, typically by using the *clflush* instruction which flushes a certain cache line from the LLC and, thus, also from lower level caches. While the locations where bit flips occur are random, they are also highly reproducible, as the row and the row offset at which a bit flip occurs will not change.

There are three different patterns in which memory accesses can be performed to trigger the Rowhammer bug. These different methods vary in complexity as well as in reliability.

- **Double-sided hammering:** To perform double-sided hammering, the attacker chooses a target row in which the bit flips should occur. Next, the two addresses which are mapped to neighboring rows in DRAM are calculated using the DRAM mapping function. These two addresses are then accessed in an alternating pattern and flushed from the cache after each access, thus, “hammering” the memory bank in which the bit flips shall occur. If the DRAM module is susceptible to Rowhammer, bit flips will occur in different rows in the memory bank under attack and possibly in the row selected by the attacker. This is by far the most complex variant of Rowhammer attacks, as the attacker has to know the mapping from physical addresses to DRAM addresses. This mapping can be reverse-engineered, as described in Section 4.1. While a Rowhammer attack using this access pattern is harder to perform than attacks using the other two patterns, it also leads to the best results regarding the amount of produced bit flips over time.

- **Single-sided hammering:** With single-sided hammering, no knowledge about the mapping from physical to DRAM addresses has to be acquired. This access pattern does not actually read from only one memory address, as the name might suggest, but from eight randomly selected addresses. With this access pattern, bit flips are less likely to occur than with double-sided hammering. The main advantage of single-sided hammering is that no knowledge about the DRAM mapping function is needed.

Single-sided and Double-sided hammering were both initially proposed by Seaborn and Dullien [4].

- **One-location hammering:** This method of hammering was initially introduced by Gruss et al. [15] and is by far the simplest possible access pattern triggering the Rowhammer bug. Here, the pattern consists of repeatedly accessing a single memory location and evicting it from the cache after each access. As described above, accessing a single row from a bank was always assumed to result in a row hit and not in a row conflict, implying that this access pattern should not be able to produce bit flips as the rows should not be opened and closed fast enough to drain enough charge to cause bit flips. While this might be true for an open-row policy or an adaptive row policy, a closed row policy will produce effects similar to the ones achieved when producing row conflicts as the targeted row has to be re-opened at every access, thus causing bit flips in the bank in which the memory accesses are performed. This method of hammering can also be successful if an adaptive row policy is used given that certain premises are fulfilled, as will be discussed in Section 5.

Rowhammer relies on memory accesses served directly from DRAM as bit flips will only occur if rows are closed and opened fast enough. Thus, it is essential to evict accessed addresses from the cache. To achieve this effect, two possible methods of removing the target address from the cache are available.

Eviction with *clflush*: Here, the cache line in which the accessed address is stored is entirely flushed by an explicit call to the *clflush* instruction [16]. This command can be called after every memory access to guarantee that the subsequent access will be served from DRAM again. This method of cache eviction was used in the initial Rowhammer variant proposed by Kim et al. [3].

Eviction with an eviction set: This method requires partial knowledge of the function mapping addresses to cache sets. Using this knowledge, one can construct a set of addresses which will evict the target address from the cache, thus enabling the attacker to perform a Rowhammer attack without explicit use of the *clflush* instruction [5]. The main disadvantage of this method is that there are additional memory accesses necessary to evict the target address from the cache, thus, reducing the frequency of possible read accesses from the target address.

Performing a Rowhammer attack: If one wants to perform an attack using Rowhammer, one can profile the DRAM for rows in which bit flips occur and also at which offset in the row these flips will occur. This information can then be used to perform an attack in which a single bit flip is enough to gain root privileges as the location of bit flips is highly reproducible. For this, the vulnerable memory has to be filled with a data structure in which a single bit flip is enough to alter the program flow in the attacker’s favor. In Seaborn’s

exploit [4], the memory was sprayed with page table entries in the preparation phase of the attack in which the induced bit flips could be abused to gain kernel privileges.

A different approach was introduced by Gruss et al. [15] with opcode-flipping, in which the attacker tries to induce bit flips in a user program, such that an unprivileged user process is able to gain root privileges.

Xiao et al. [17] demonstrated that Rowhammer could be used to break memory isolation enforced by virtualization on servers, thus, showing that this class of attacks is not limited to client-side bit flips.

3 Test Setup

In the following sections, different tests and attacks were performed on one or more of the available laboratory setups. This section gives an overview of the three different machines that were used in the experiments.

Lab 02 was used to perform simple Rowhammer tests beforehand and to flood the Nethammer target with UDP packages. It was also used to evaluate the correct functionality of the modified DRAMA tool and the row inspector tool.

Lab 06 was used to test if the RAM used in the target configuration is vulnerable to Rowhammer. This step was performed on this machine as the DRAM mapping function for this machine was already known. This device was also used to flood the target during the Nethammer experiment.

Lab 08 was the target computer for the Nethammer experiment. This machine was used to perform all tests regarding Rowhammer in combination with Intel CAT and also Nethammer.

Name	Processor	Clock	L3 Cache size	DRAM
Lab 02	Haswell i7-4790	3.6 GHz	8 MB	16 GB DDR3
Lab 06	Skylake i7-6700K	4.0 GHz	8 MB	16 GB DDR4
Lab 08	Broadwell-EP Xeon E5-1630 v4	3.7 GHz	10 MB	16 GB DDR4

Table 1: Laboratory setup

In Table 1 we provide the technical specifications of the three devices.

4 Reverse-Engineering Properties of DRAM and Memory Controllers

In this section we discuss how we gathered information about the DRAM and the memory controller of the system, with which the Rowhammer attack over the network was performed. In the first part of this section, we cover the reverse-engineering of the DRAM mapping function which was required to perform double-sided hammering as described in Section 5. In the second part of this section, we elaborate on the workings of the *row inspector*, a tool developed to deduce the row policy of the system under test.

4.1 Reverse-Engineering of DRAM Mappings

To perform double-sided hammering, the mapping from physical addresses to DRAM addresses has to be known. This undocumented mapping function depends on the processor’s memory controller, the number of processor’s and the number of DIMMs in the system. It is obtained by using the reverse-engineering technique by Pessl et al. [1] for each new combination of components. The basic idea of this approach is the identification of *conflicting addresses*, by measuring the timings of subsequent accesses. As addresses mapping to the same bank will generally produce a row conflict on subsequent accesses, a timing difference between accesses to conflicting (i.e., mapping to the same bank) and non-conflicting addresses can be measured. By observing the timings of different address pairs, one can generate sets of conflicting addresses from which a mapping function can be derived by solving a linear equation system.

While this approach may work well for conventional systems using an open- or closed-row policy, there are some problems which may occur if an adaptive row policy is used. As described in Section 2, memory controllers which use an adaptive row policy aim to reduce the latency of memory accesses by dynamically adjusting the time a row is kept open after each access. This greatly affects the timings measured by the reverse-engineering gadget as conflicting addresses will produce enough row conflicts to cause a change from closed- to open-row policy during the measurement phase. To bypass this interference, we reduced the number of subsequent accesses to the address pairs such that the memory controller does not apply the adaptive policy (i.e. does not change to the closed row policy). With this modification, we were able to get reproducible measurement results on the system under test.

Another source of disturbance can arise from a poorly conditioned set of conflicting addresses. If all of the randomly selected addresses have certain bits in common which are not influencing the mapping, those bits will still be found to be part of the mapping function erroneously. If the amount of addresses per set is low, this can cause false positives in the calculation of the mapping function, thus, producing unusable results. The solution to this problem is to monitor the number of bits which are equal in all addresses of a set and to use this information to attempt to find addresses which fit in the set but break the pattern of equal bits, thus eliminating errors induced by the random selection of addresses.

A further effective method to counter this source of error is to increase the number of addresses that each set should contain, thus reducing the probability that bits are equal in all addresses of a set due to coincidence.

To validate the correctness of these improvements, the modified tool was used to reverse-engineer the mapping on Lab 02 and Lab 06. The results of the modified version were found to be correct, as they matched the functions obtained by Pessl et al. [1] for the same configurations.

Using the unmodified tool, we were not able to obtain a correct mapping function for the configuration used in Lab 08 due to the adaptive row policy. Hence, the modifications mentioned above were introduced in the tool. With the modified tool it was possible to correctly reverse-engineer the mapping function depicted in Figure 2, and subsequently use it in the double-sided Rowhammer tests described in Section 5.

Note that the labels assigned in Figure 2 are chosen arbitrarily, meaning

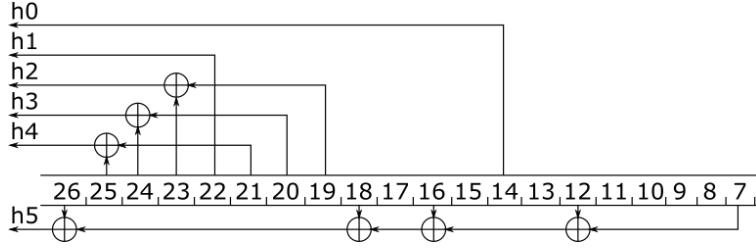


Figure 2: Reverse-engineered mapping function for DRAM addresses on Lab 08

that the order of XOR'd bits does not influence the results obtained when using this or any other mapping function.

4.2 Reverse-Engineering of Memory Controller Behaviour

In this section, we discuss the process of reverse-engineering the row policy used by the memory controller of a target system and, if an adaptive policy is used, when the memory controller decides to modify row timings. This step was necessary as the results produced by the reverse-engineering tool described in Section 4.1, and the results of the single-location Rowhammer test on Lab 08 were not usable while the results obtained on Lab 02 and Lab 06 were just as expected.

To solve this problem, we created the *row inspector* tool, a tool to visualize the access timings to a single address depending on the number of previous accesses.

The basic idea of this tool is the measurement of timings in memory accesses to one single address after accessing the address for a variable amount of times beforehand. Given a single address A and assuming that the memory controller initially utilizes a closed-row policy, we want to find n such that n consecutive accesses to the row in which A is mapped to will convince the memory controller to switch to an open-row policy. As these accesses have to reach DRAM and not the cache, it is essential that the cache line containing A is flushed after every access. It is also important that memory accesses are not re-ordered in any way, hence, an *mfence* [18] instruction has to be added after accessing the target address. The *mfence* instruction serializes all previously issued load and store instructions, thus, enabling us to access the target memory location without any re-ordering. By varying the number of accesses to A before the actual measurement, it is possible to find the number of accesses needed to force the memory controller into switching row policies. We expect a notable timing difference between the open and the closed row policy, as all measured accesses will be row hits after the policy has changed. The initial code of the tool is shown in Listing 1.

```

while(initial_accesses-- > 0)
{
    *A;
    asm volatile("mfence");
    asm volatile("clflush(%0)":"::r"(A):"memory");
    asm volatile("mfence");
}
t1 = rdtsc();    //Get Timestamp 1
*A;
t2 = rdtsc();    //Get Timestamp 2
asm volatile("clflush(%0)":"::r"(A):"memory");
asm volatile("mfence");
return t2-t1;

```

Listing 1: Basic code of the row inspector gadget.

With this approach, we got measurements where the lowest access timings were too low for conventional DRAM accesses, regardless of the utilized policy. As this should not be possible, we concluded that there had to be a factor which was not accounted for in the initial code. After some testing, we found that the reason for these false measurements was branch prediction in the while-loop. If the number of initial accesses is sufficient, branch prediction will speculatively execute the most likely branch. This causes the memory access to A to be performed even if the condition in the loop header evaluates to false, thus, caching A and causing the subsequent measured access to be served from the cache and not from DRAM. This disturbs the measurement process and generates unusable results.

A straightforward fix to this problem is the introduction of an additional cache flush after the loop, thus, leading to the code presented in Listing 2.

```

while(initial_accesses-- > 0)
{
    *A;
    asm volatile("mfence");
    asm volatile("clflush(%0)":"::r"(A):"memory");
    asm volatile("mfence");
}
asm volatile("clflush(%0)":"::r"(A):"memory");
asm volatile("mfence");
t1 = rdtsc();    //Get Timestamp 1
*A;
t2 = rdtsc();    //Get Timestamp 2
asm volatile("clflush(%0)":"::r"(A):"memory");
asm volatile("mfence");
return t2-t1;

```

Listing 2: Final code of the row inspector gadget, modified for branch prediction.

With this modified variant, it was possible to reverse-engineer the behavior of the memory controller regarding the adaptive row policy, as shown in Figure 3. The obtained results helped explaining why the modifications to the DRAMA

tool mentioned in Section 4.1 were indeed improving the gathered results. By reducing the number of memory accesses, the memory controller’s policy adaption was avoided, thus, helping to perform undisturbed timing measurements which can be used to generate the DRAM mapping functions.

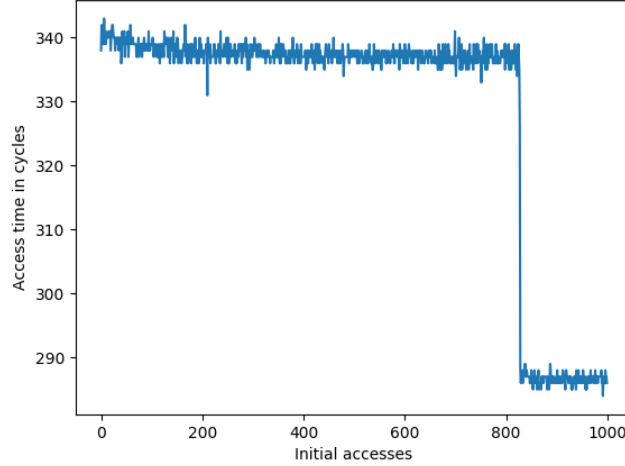


Figure 3: Measurement of Lab 08 memory controller row policy

While the row policy can be detected on a system with low memory load, we were not able to reproduce our measurements on systems exposed to a certain memory stress. It seems that memory accesses done by web browsers or media players are sufficient to convince the memory controller to use its standard closed-row policy.

The results obtained using this tool greatly helped to understand the outcomes of the different Rowhammer tests discussed in the following section.

5 Evaluation of different Rowhammer patterns

To mount a Rowhammer attack over the network, we initially had to verify whether the system under test is vulnerable to Rowhammer at all. This section discusses the results obtained with regular Rowhammer attacks, namely with double-sided and one-location hammering and the modifications that were required to produce bit flips.

The code used for the tests discussed in this section still uses the *clflush* instruction to evict accessed addresses from the cache.

To check if the system is vulnerable to Rowhammer, three different hammering patterns are considered.

5.1 Double-sided Hammering

According to previous research, the expected amount of bit flips is highest when using double-sided hammering [15]. As discussed in Section 2, the mapping from

physical addresses to DRAM addresses has to be known to mount a double-sided Rowhammer attack, as this kind of attack relies on producing row conflicts by alternately accessing two addresses mapping to the same bank in DRAM. The test was performed using the DRAM mapping function we reverse-engineered as described in Section 4.2. With this mapping function, no bit flips could be observed.

Assuming that the mapping function that we reverse-engineered was indeed correct, another factor had to be the source for the lack of bit flips in this test. The processor used in the system under test has TRR [9] implemented in the memory controller. With TRR, the row under attack is identified by the memory controller and subsequently refreshed more frequently than other rows, thus, mitigating the effect that hammering has on this row. While this technique helps to avoid bit flips in a certain target row, we show that it does not eliminate bit flips in rows which are not located between the two hammered rows.

To bypass TRR, we modified the scan-phase of the attack such that non-adjacent rows are also scanned for bit flips. After performing this modification, several bit flips were detected in non-adjacent rows within seconds.

This proves that TRR does not mitigate Rowhammer attacks at all. TRR only prevents bit flips in the detected target row, but not in other rows in the bank, thus, barely reducing the attack surface. It is not required to cause bit flips in a specific row to exploit them. Reproducible bit flips in rows which are not located between the two aggressor rows are enough to mount a regular Rowhammer attack.

These results showed that the system under test was susceptible to Rowhammer and that the mapping function obtained with the modified DRAMA-tool was indeed correct.

5.2 One-location hammering

For one-location hammering, we did not need the mapping function as only a single memory row is accessed, and no location has to be known. We used the same code which was previously used by Gruss et al. [15]. This code allocates a large chunk of memory (the size can be specified by the user) and performs a large number of subsequent accesses to random addresses, thus, effectively hammering them. After a certain amount of addresses hammered, the memory is scanned for bit flips.

Using this code, we were not able to produce any bit flips on the targeted system. As a single address is accessed roughly 5 million times, one can assume that the row policy of the memory controller will adapt to such behavior by keeping the targeted row open instead of closing it. This effectively mitigates the Rowhammer attack, as no row conflicts occur, as discussed in Section 2.

While the adaptive row policy can prevent the one-location hammering on a system with no other kind of memory stress, except for the application performing the attack, it does not mitigate the attack if other applications are performing a sufficient amount of memory accesses. During reverse-engineering of the memory controller policy, we found that the adaptive policy is only activated if the memory noise on the system is kept low.

By generating memory accesses from other sources (the *stress* command, or a web browser), we were able to hinder the memory controller from switching

row policies, thus convincing it to keep using its closed-row policy. Hence, bit flips can be observed with the used code.

In conclusion, we were able to confirm that the system Lab 08 is indeed susceptible to Rowhammer attacks using *clflush*, if certain conditions are met. For a successful double-sided attack, TRR was bypassed by checking for bit flips in non-adjacent rows. In order to perform a one-location hammering attack, additional memory stress had to be generated on the system to avoid the memory controllers adaptive row policy. As both tests produced a positive result, we were able to confirm that the system under test is vulnerable to Rowhammer attacks.

6 Evaluation of Rowhammer with Intel CAT

In this Section, we discuss the steps necessary to transform the *clflush*-based Rowhammer attacks presented in Section 2.4 and evaluated in Section 5 into attacks which do not rely on explicit cache flush instructions by utilizing CAT. As described in Section 2.3, Intel CAT splits the L3 cache into several classes of service, thus, effectively reducing the available last-level cache for each process. Assuming that an attacker knows the function used to map addresses to cache slices, one can evict addresses from the cache with high precision as has been previously shown and used to perform attacks [10, 15]. As the cache size is limited by the capacity mask of the CLOS when using CAT, one can easily construct a reduced eviction set for a given address, thus, minimizing the number of memory accesses required to evict this address from the cache. Using these findings in combination with the Rowhammer code used in Section 5, we were able to create two Rowhammer attacks which do not rely on *clflush* to evict the aggressor address from the cache while performing equally well as conventional attacks.

Before performing the following tests, Intel CAT had to be set up on the system under test such that the desired cache allocation could be performed. For the following attack scenarios, the attacker code was pinned to a processor core whose class of service was configured such that only one cache way per set was available.

6.1 Double-sided Hammering with CAT

To perform double-sided hammering with CAT, the code used in Section 5 was modified such that no explicit *clflush* instruction is used to evict the two aggressor addresses from the cache. Instead, we selected two additional addresses, one for each aggressor, which are used as the eviction set for the respective address. Given two addresses f and s which map to the same DRAM bank, and two additional addresses $f2$ and $s2$ which map to the same cache slice as their respective aggressor address, one can perform a double-sided Rowhammer attack by accessing them as shown in Listing 3.


```

while (number_of_reads- > 0)
{
    *f;           //Access first aggressor
    *f2;          //Evict first aggressor from cache
    *s;           //Access second aggressor
    *s2;          //Evict second aggressor from cache
}

```

Listing 3: Modified code for double-sided hammering

As with the double-sided attack in Section 5.1, we still had to bypass TRR by checking not only adjacent rows but also other rows in the same bank as the target addresses. For *f2* and *s2*, no additional conditions have to be met as they are not used for hammering but for eviction.

By changing the code as described above, we were able to generate a slightly smaller amount of bit flips than with the native code in Section 5.1 if CAT was active on the core which executed the program. This shows that *clflush* is not required to perform a double-sided Rowhammer attack if the system utilizes CAT to reduce the L3 cache size for the hammering process.

6.2 One-location Hammering with CAT

As the last test, one-location hammering was adapted such that no cache flush instruction is needed. As in the flush-free double-sided hammering test, all *clflush* instructions were removed from the code. Instead, a single eviction address was selected in order to evict the aggressor address from the cache after each access. The code is shown in Listing 4.

```

while (number_of_reads- > 0)
{
    *f;           //Access aggressor
    *f2;          //Evict aggressor from cache
}

```

Listing 4: Modified code for one-location hammering

Interestingly, this method of hammering still relies on additional memory stress on the system in order to produce bit flips. Running this code on two cores in parallel will lead to a fair amount of bit flips. The same holds for tests where additional memory stress is generated with *stress -m 1*.

These two tests confirmed that by using Intel CAT, we are able to perform Rowhammer attacks without using the *clflush* instruction. Aggressor addresses can be flushed by a single additional memory access as the cache size is strongly limited by CAT, enabling us to mount Rowhammer attacks with performance comparable to conventional attacks which rely on manually flushing the cache.

Comparing the results of Rowhammer with and without Intel CAT, one can see that CAT can improve the probability of bit flips in the case of one-location hammering, as shown in Figure 4.

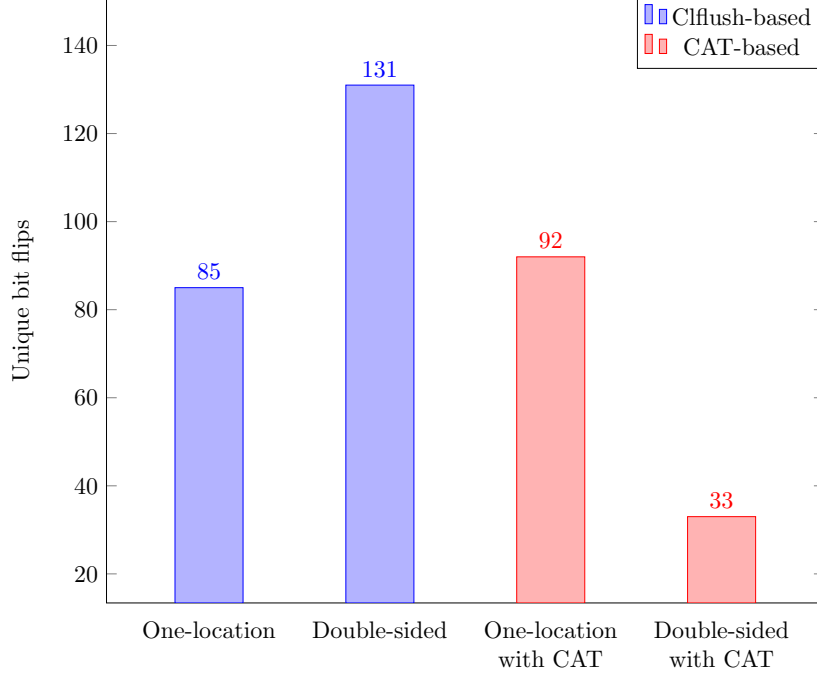


Figure 4: Evaluation of different Rowhammer patterns on Lab 08

7 Nethammer

Based on the conclusions of the previous sections, we now mount a new attack dubbed *Nethammer*, in which the bit flips are induced over the network. The benefit of such an attack is, that no line of attacker code needs to be executed on the target machine, enabling remote attacks which are hard to trace as no file containing malicious code can be found on the system. Given that we do not want any external code to run on the system, we had to find code on the target system which would act as our remote Rowhammer agent. We found that some code handling incoming network packets (i.e., the network driver) could be suitable for this task.

7.1 Initial Considerations

Given that this is a new attack setup, some considerations regarding the achievable memory load have to be done. As the average memory access measured in Section 5 took approximately 280 cycles at a clock frequency of 3.7 GHz (see Table 1), the duration of an average memory access is 76 ns. From the duration of one memory access, we can deduce that, in our previous attacks, approximately 13 000 000 memory accesses were performed per second. If we manage to create a number of accesses comparable to the one in regular Rowhammer attacks, bit flips should occur. This indicates that we need to send a very large amount of network packages to the system under attack. To achieve this goal, we use UDP packets containing a single 8-bit character, resulting in a packet size of 64 Bytes.

Using a 1 GBit connection, it is possible to send up to almost 2 000 000 such packets per second; this amount is already in the same order of magnitude as the number of memory accesses performed by local Rowhammer code. Assuming that the network driver performs several memory accesses per received packet, it should be possible to abuse it to induce bit flips, as the expected amount of memory accesses will then be sufficiently high.

The above assumptions require that CAT is activated on the system and, thus, causing memory accesses to be served from DRAM more frequently.

7.2 Attack Setup and Evaluation

Attacker 1 and attacker 2 were configured such that the target machine was constantly flooded with UDP packets containing a single 8-bit character over a 1 GBit Ethernet switch, as depicted in Figure 5.

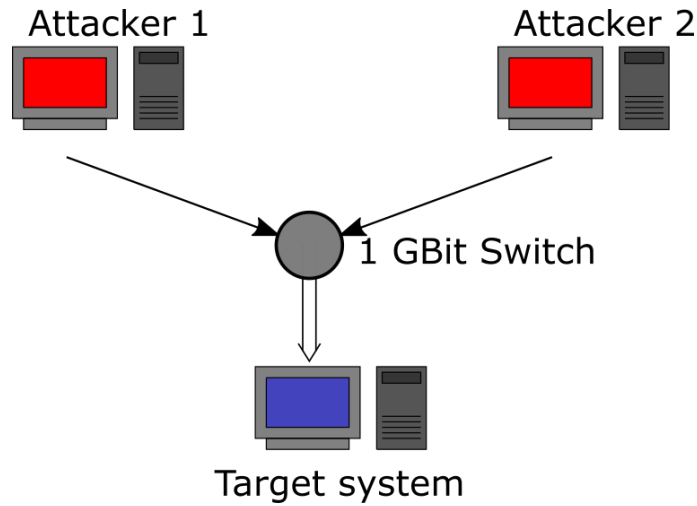


Figure 5: Schematic setup for the Nethammer attack

On the target system, Intel CAT was configured such that the core on which network interrupts are handled is limited to one cache way in order to serve memory accesses from DRAM more frequently. The tests were performed with two 8 GB DDR4 modules, adding up to 16 GB of DRAM.

A large portion of the target systems memory was continuously scanned for potential bit flips. The scanning method relied on filling large sections of the system’s DRAM with recognizable patterns and scanning for bitwise deviations in said patterns. Such deviations can only be credited to bit flips given that the content of DRAM is not modified by the scanning tool itself.

Once the target was flooded with UDP packages, a significant increase of cache misses on the core handling network packets was detected, we registered approximately 25.000.000 cache misses per second, thus, proving our assumption that several memory accesses are performed per network package. Such a high number of cache misses indicates a large number of memory accesses served from DRAM, suggesting that the network driver can indeed be used as a hammering agent on the target system. With no additional load on the system, we detected

one bit flip within 12 hours. As soon as we increased the local load on the system, we found more bit flips, as can be seen in Figure 6. The measurements

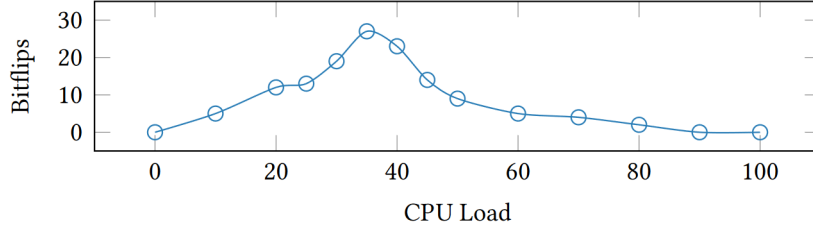


Figure 6: Bit flip count in relation to CPU load [2]

performed show, that the amount of bit flips strongly depends on the system load, peaking at approximately 35%.

Our attack proves that it is possible to remotely induce bit flips in a target system by sending network packets at sufficient speeds, given that the system under attack uses Intel CAT.

8 Conclusion

In this work, we proved that it is possible to induce bit flips remotely by flooding a target machine with network packets, given the premise that the target machine uses Intel CAT. We performed the attack in a laboratory setup with three machines, two attackers and one target. Configuring Intel CAT on the target, such that the core handling the network packages is limited to one cache way, caused several bit flips to occur in the scanned memory area.

Aside from the remote attack, we also found that CAT enabled us to perform local Rowhammer attacks without explicit calls to the cache flush function. The amount of bit flips obtained in the experiments with local code execution and CAT was comparable to the amount of bit flips observed when using *clflush*, indicating that a reduction of the cache size facilitates flush-free Rowhammer attacks.

During the preparation for the attacks mentioned above, we also created a tool with which we were able to reverse-engineer the memory controllers row policy by measuring memory access timings. With this tool we were able to test how different memory loads affect the adaptive row policy, leading to the result that increasing memory load will stop the memory controller from changing the policy. We found that, even if an adaptive policy was utilized, Rowhammer attacks were still feasible, if the additional memory load on the system is high enough.

We also improved the stability of the DRAM reverse-engineering tool by conditioning the address sets such that false positives through bits which are set in all addresses were prevented.

References

- [1] P. Pessl, D. Gruss, C. Maurice, M. Schwarz, and S. Mangard, “DRAMA: Exploiting DRAM Addressing for Cross-CPU Attacks,” in *USENIX Security Symposium*, 2016.
- [2] M. Lipp, M. T. Aga, M. Schwarz, D. Gruss, C. Maurice, L. Raab, and L. Lamster, “Nethammer: Inducing Rowhammer Faults through Network Requests,” *arXiv preprint arXiv:1805.04956*, 2018.
- [3] Y. Kim, R. Daly, J. Kim, C. Fallin, J. H. Lee, D. Lee, C. Wilkerson, K. Lai, and O. Mutlu, “Flipping bits in memory without accessing them: An experimental study of DRAM disturbance errors,” in *ACM SIGARCH Computer Architecture News*, vol. 42, pp. 361–372, IEEE Press, 2014.
- [4] M. Seaborn and T. Dullien, “Exploiting the DRAM Rowhammer bug to gain kernel privileges,” *Black Hat USA*, 2015.
- [5] D. Gruss, C. Maurice, and S. Mangard, “Rowhammer.js: A remote software-induced Fault Attack in JavaScript,” *DIMVA*, 2016.
- [6] Z. B. Aweke, S. F. Yitbarek, R. Qiao, R. Das, M. Hicks, Y. Oren, and T. Austin, “ANVIL: Software-based protection against next-generation Rowhammer attacks,” *ACM SIGPLAN Notices*, vol. 51, no. 4, pp. 743–755, 2016.
- [7] F. Brasser, L. Davi, D. Gens, C. Liebchen, and A.-R. Sadeghi, “Can’t touch this: Software-only mitigation against Rowhammer attacks targeting kernel memory,” in *USENIX Security Symposium*, 2017.
- [8] M. T. Aga, Z. B. Aweke, and T. Austin, “When good protections go bad: Exploiting anti-DoS measures to accelerate Rowhammer attacks,” in *IEEE HOST*, 2017.
- [9] K. Bains, J. Halbert, C. Mozak, T. Schoenborn, and Z. Greenfield, “Row hammer refresh command,” Aug. 25 2015. US Patent 9,117,544.
- [10] C. Maurice, N. Le Scouarnec, C. Neumann, O. Heen, and A. Francillon, “Reverse Engineering Intel Complex Addressing Using Performance Counters,” in *RAID*, 2015.
- [11] JEDEC Solid State Technology Association and others, “JEDEC Standard: DDR4 SDRAM,” *JESD79-4*, 2012.
- [12] JEDEC Solid State Technology Association and others, “JEDEC Standard: DDR3 SDRAM,” *JESD79-3*, 2012.
- [13] Intel Corporation, “Intel 64 and IA-32 architectures optimization reference manual,” 2009.
- [14] A. Herdrich, E. Verplanke, P. Autee, R. Illikkal, C. Gianos, R. Singhal, and R. Iyer, “Cache QoS: From concept to reality in the Intel® Xeon® processor E5-2600 v3 product family,” in *High Performance Computer Architecture (HPCA), 2016 IEEE International Symposium on*, 2016.

- [15] D. Gruss, M. Lipp, M. Schwarz, D. Genkin, J. Juffinger, S. O’Connell, W. Schoecl, and Y. Yarom, “Another flip in the wall of Rowhammer defenses,” *arXiv:1710.00551*, 2017.
- [16] S. Palanca, S. A. Fischer, and S. Maiyuran, “CLFLUSH micro-architectural implementation method and system,” Apr. 8 2003. US Patent 6,546,462.
- [17] Y. Xiao, X. Zhang, Y. Zhang, and R. Teodorescu, “One Bit Flips, One Cloud Flops: Cross-VM Rowhammer Attacks and Privilege Escalation.,” in *USENIX Security Symposium*, 2016.
- [18] S. Palanca, S. A. Fischer, S. Maiyuran, and S. Qawami, “MFENCE and LFENCE micro-architectural implementation method and system,” Nov. 18 2003. US Patent 6,651,151.